

**INSTITUTO
FEDERAL**

Santa Catarina

Câmpus
São José

Comparando sockets UDP e TCP

Redes de computadores - Laboratório 07

Curso: Engenharia de Telecomunicações
Professor: Odilson Tadeu Valle

Aluno:
Victor E. de L. Guerra

03/10/2025

Sumário

1	Objetivos	2
2	Programação de sockets com TCP	2
2.1	Em novos terminais do Cliente 1 e 2 e Servidor, verifique os sockets TCP ativos	2
2.1.1	Você encontrou sockets abertos em todos os hosts (máquinas)?	2
2.1.2	Quais parâmetros apresentados para cada um deles?	3
2.1.3	Qual a relação entre os socktes clientes e servidor, número IP, portas etc? . . .	3
2.1.4	Identifique e "printe"o socket receptivo no Servidor.	4
2.2	Perguntas	4
2.2.1	Para cada cliente, as três primeiras mensagens trocadas apresentam a camada de aplicação, sim ou não? Explique. O que elas significam (3-way handshake)?	4
2.2.2	Encontre a frase/palavra escrita enviada ao servidor (minúscula) e a resposta em maiúscula?	4
2.2.3	Qual o tamanho, dos pacotes contendo as mensagens: i) Data (camada 5) e ii) Total Length (camada 3). Qual a relação entre estes valores?	5
2.2.4	As última 3 mensagens, de cada cliente, contém o fechamento de conexão, explique-as.	5
2.2.5	Qual é o protocolo da camada de transporte nessa troca de mensagens? . . .	5
2.2.6	Qual o número identificador de protocolo TCP no pacote IP? Dica: na janela central abra o campo Internet Protocol e procure a string Protocol.	6
3	Programação de sockets com UDP	6
3.1	Verifique os sockets em um novo terminal das máquinas Cliente 1 e 2 e Servidor, com o comando: ss -ua	6
3.1.1	Identifique e anote os sockets abertos.	6
3.1.2	Identifique e anote o socket receptivo do servidor.	7
3.2	PERGUNTAS baseadas na captura:	7
3.2.1	Em algum momento foi identificado algum procedimento para estabelecimento de conexão?	7
3.2.2	Em algum campo do UDP existe numeração de mensagens?	8
3.2.3	Qual o número identificador de protocolo UDP no pacote IP?	8
3.2.4	Qual é o checksum no pacote (datagrama) UDP? Qual é o formato apresentado? Quantos bits ele possui?	8
3.2.5	É possível capturar toda a troca de mensagens e inclusive capturar o texto passado do cliente para o servidor?	8
3.2.6	Qual foi a sequência numérica do campo Data em seu teste? Qual o significado?	8
3.2.7	Qual é o protocolo da camada de transporte nessa troca de mensagens? . . .	9
4	Comparativo entre UDP e TCP	9
4.1	Quantas mensagens foram trocadas entre o servidor e o cliente em cada um dos protocolos para atingir o mesmo objetivo?	9
4.2	O que justifica a diferença na quantidade de mensagens trocadas?	9
4.3	Discuta as vantagens e desvantagens de cada protocolo	9
4.3.1	UDP	9
4.3.2	TCP	10

1 Objetivos

- Entender o conceito de sockets relacionados aos protocolos UDP e TCP.
 - Processos que rodam em máquinas diferentes se comunicam entre si enviando mensagens para sockets. Um processo é semelhante a um prédio e o socket do processo é semelhante a uma porta em seu interior. A aplicação reside dentro do prédio e o protocolo da camada de transporte reside no mundo externo. Um programador de aplicação controla o interior do prédio mas tem pouco (ou nenhum) controle sobre o exterior.
- Simultaneamente explora-se os conceitos relativos aos protocolos UDP e TCP, observando-se a quantidade de mensagens necessárias para a troca de uma simples frase textual.
 - Observa-se a "agilidade" do UDP e a robustez do TCP.
- Por fim, propõe-se um comparativo entre os dois protocolos da camada de transporte: UDP e TCP.

2 Programação de sockets com TCP

2.1 Em novos terminais do Cliente 1 e 2 e Servidor, verifique os sockets TCP ativos

2.1.1 Você encontrou sockets abertos em todos os hosts (máquinas)?

Sim, haviam sockets abertos em todos os hosts, como pode-se ver nas imagens abaixo.



The image shows a terminal window titled "IMUNES: Servidor (console) bash". The user is at the root prompt on the server. They run the command `ss -nta | grep -E '33333|Local'`. The output shows the following:

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
LISTEN	1	1	0.0.0.0:33333	0.0.0.0:*	
ESTAB	0	0	10.0.0.10:33333	10.0.0.21:44190	
ESTAB	0	0	10.0.0.10:33333	10.0.0.20:41160	

The terminal prompt is `root@Servidor:/#`.

Figura 1: Sockets abertos no servidor

```
IMUNES: Cliente1 (console) bash
root@Cliente1:/# ss -nta | grep -E '33333|Local'
State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
ESTAB 0      0      10.0.0.20:41160    10.0.0.10:33333
root@Cliente1:/#
```

Figura 2: Sockets abertos no cliente 1

```
IMUNES: Cliente2 (console) bash
root@Cliente2:/# ss -nta | grep -E '33333|Local'
State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
ESTAB 0      0      10.0.0.21:44190    10.0.0.10:33333
root@Cliente2:/#
```

Figura 3: Sockets abertos no cliente 2

2.1.2 Quais parâmetros apresentados para cada um deles?

Como apresentado nas imagens 1, 2 e 3, podemos ver que os parâmetros para cada socket são: *State*, *Recv-Q*, *Send-Q*, *Local Address:Port*, *Peer Address:Port* *Process*

2.1.3 Qual a relação entre os socktes clientes e servidor, número IP, portas etc?

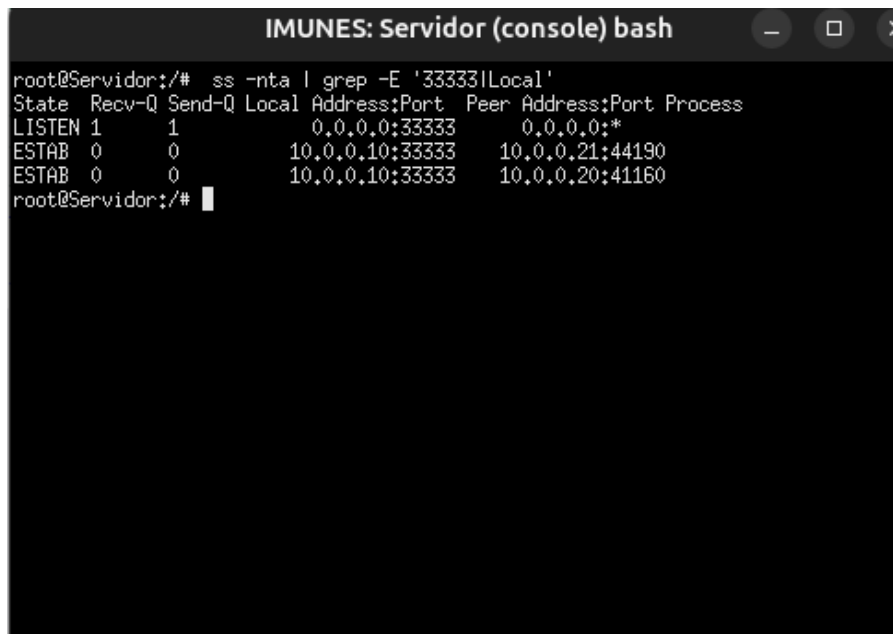
No servidor, as informações de *Local Address* e porta correspondem ao endereço IP do próprio servidor e à porta previamente configurada para que os sockets escutem conexões — neste caso, a porta 33333. Já as informações de *Peer Address* e *Port Process* indicam o endereço IP do respectivo cliente que estabeleceu a conexão e a porta efêmera (escolhida automaticamente pelo sistema operacional) utilizada por ele.

Para os clientes, há apenas um socket associado à aplicação que se conecta ao servidor. Nesse caso, o *Local Address* e a porta correspondem ao IP do cliente e a uma porta efêmera escolhida

automaticamente, enquanto o *Peer Address* e o *Port Process* são, respectivamente, o IP do servidor e a porta fixa 33333.

2.1.4 Identifique e "printe" o socket receptivo no Servidor.

Como podemos ver na imagem 4, vemos que o socket receptivo é o que começa com "Listen".



```
IMUNES: Servidor (console) bash
root@Servidor:/# ss -nta | grep -E '33333|Local'
State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
LISTEN 1      1      0.0.0.0:33333      0.0.0.0:*
ESTAB  0      0      10.0.0.10:33333    10.0.0.21:44190
ESTAB  0      0      10.0.0.10:33333    10.0.0.20:41160
root@Servidor:/#
```

Figura 4: Socket receptivo

2.2 Perguntas

2.2.1 Para cada cliente, as três primeiras mensagens trocadas apresentam a camada de aplicação, sim ou não? Explique. O que elas significam (3-way handshake)?

Não. As três primeiras mensagens trocadas entre cliente e servidor correspondem ao *three-way handshake* do TCP, que ocorre na **camada de transporte**. Esse processo garante o estabelecimento de uma conexão confiável antes que qualquer dado da camada de aplicação seja enviado. As mensagens são:

- **SYN**: o cliente solicita a abertura de uma conexão.
- **SYN+ACK**: o servidor confirma a solicitação e propõe a sincronização de sequência.
- **ACK**: o cliente confirma a sincronização, completando o *handshake*.

2.2.2 Encontre a frase/palavra escrita enviada ao servidor (minúscula) e a resposta em maiúscula?

A mensagem enviada pelo cliente é uma **frase em minúsculas**, encaminhada como dados da aplicação (camada 5). O servidor responde com a mesma frase transformada em **maiúsculas**. Exemplo:

"aula de redes-1" → "AULA DE REDES-1"

2.2.3 Qual o tamanho, dos pacotes contendo as mensagens: i) Data (camada 5) e ii) Total Length (camada 3). Qual a relação entre estes valores?

Nas trocas que contêm dados, é possível observar:

- **Len (camada 5 - dados da aplicação):** representa a quantidade de bytes efetivamente transportados pela aplicação.
- **Total Length (camada 3 - IP):** representa o tamanho total do datagrama IP (cabeçalho IP + cabeçalho TCP + dados da aplicação).

A relação entre esses valores é:

$$\text{Total Length (IP)} = \text{Cabeçalho IP} + \text{Cabeçalho TCP} + \text{Dados da aplicação}$$

Ou seja, o campo de dados (camada 5) é apenas uma parte do tamanho total do pacote na camada de rede.

Abaixo temos imagens com essas informações:

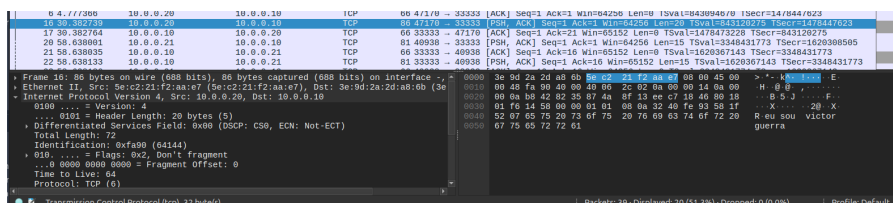


Figura 5: Imagem do Total length

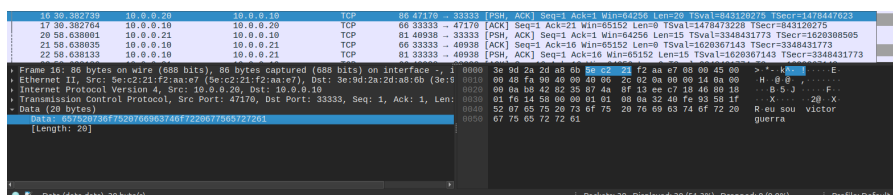


Figura 6: Imagem do length (tamanho) apenas das informações na camada de aplicação

2.2.4 As última 3 mensagens, de cada cliente, contém o fechamento de conexão, explique-as.

O término da conexão TCP ocorre em um processo chamado **four-way handshake**, onde cada lado encerra sua metade da comunicação. No trace capturado, as últimas três mensagens observadas são:

- **FIN, ACK (servidor):** o servidor indica que não enviará mais dados.
- **FIN, ACK (cliente):** o cliente confirma e também sinaliza que não enviará mais dados.
- **ACK (servidor):** o servidor confirma o término da conexão.

Esse processo garante que os dois lados concordam com o fechamento ordenado da sessão.

2.2.5 Qual é o protocolo da camada de transporte nessa troca de mensagens?

O protocolo utilizado na camada de transporte nesta troca de mensagens é o **TCP (Transmission Control Protocol)**, pois há estabelecimento de conexão, controle de sequência, confiabilidade e encerramento organizado.

2.2.6 Qual o número identificador de protocolo TCP no pacote IP? Dica: na janela central abra o campo Internet Protocol e procure a string Protocol.

No cabeçalho do protocolo IP (camada 3), há um campo chamado `Protocol` que indica qual protocolo da camada superior está encapsulado. O valor encontrado é:

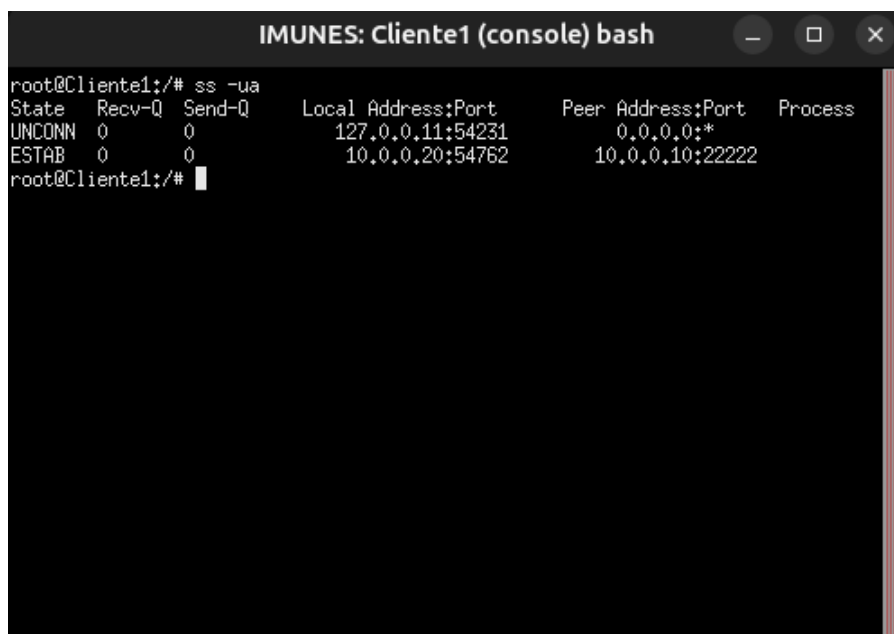
6 → TCP

Esse identificador é padrão na tabela IANA de protocolos IP.

3 Programação de sockets com UDP

3.1 Verifique os sockets em um novo terminal das máquinas Cliente 1 e 2 e Servidor, com o comando: `ss -ua`

3.1.1 Identifique e anote os sockets abertos.



The image shows a terminal window titled "IMUNES: Cliente1 (console) bash". The user has executed the command `ss -ua`. The output is as follows:

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
UNCONN	0	0	127.0.0.11:54231	0.0.0.0:*	
ESTAB	0	0	10.0.0.20:54762	10.0.0.10:22222	

Figura 7: Sockets abertos no cliente 1

```
IMUNES: Cliente2 (console) bash
root@Cliente2:/# ss -ua
State      Recv-Q    Send-Q    Local Address:Port    Peer Address:Port    Process
ESTAB      0          0          10.0.0.21:37522       10.0.0.10:22222
UNCONN     0          0          127.0.0.11:58731      0.0.0.0:*
```

Figura 8: Sockets abertos no cliente 2

3.1.2 Identifique e anote o socket receptivo do servidor.

Como visto na imagem abaixo, o socket receptivo é o `0.0.0.0:22222`, que foi a porta que configuramos no código python.

```
IMUNES: Servidor (console) bash
root@Servidor:/# ss -ua
State      Recv-Q    Send-Q    Local Address:Port    Peer Address:Port    Process
UNCONN     0          0          0.0.0.0:22222         0.0.0.0:*
UNCONN     0          0          127.0.0.11:38160      0.0.0.0:*
```

Figura 9: Socket receptivo

3.2 PERGUNTAS baseadas na captura:

3.2.1 Em algum momento foi identificado algum procedimento para estabelecimento de conexão?

O protocolo UDP é **orientado a datagrama** e não estabelece conexão antes da troca de mensagens. Nos pacotes analisados:

- Pacotes 4 e 5 mostram requisições Handshake/Ping, que indicam tentativa de comunicação, mas não configuram uma conexão formal como no TCP.

Portanto, não há um procedimento formal de estabelecimento de conexão.

3.2.2 Em algum campo do UDP existe numeração de mensagens?

O cabeçalho UDP possui os seguintes campos principais:

- Porta de origem (Source Port)
- Porta de destino (Destination Port)
- Comprimento do datagrama (Length)
- Checksum

Não existe um campo específico para numeração de mensagens, ao contrário do TCP. Ou seja, o UDP não garante ordenação nem detecção de duplicação baseada em números de sequência.

3.2.3 Qual o número identificador de protocolo UDP no pacote IP?

No cabeçalho IP, o campo `Protocol` indica o protocolo da camada de transporte. Para UDP, o identificador é:

$$\text{Protocol UDP} = 17$$

Este valor aparece na análise do campo `Protocol` do pacote IP.

3.2.4 Qual é o checksum no pacote (datagrama) UDP? Qual é o formato apresentado? Quantos bits ele possui?

O checksum UDP é utilizado para verificação de integridade dos dados. Características:

- Formato apresentado: hexadecimal (por exemplo, algo como 0x1a2b)
- Tamanho: 16 bits

O checksum cobre o cabeçalho UDP, os dados e um pseudo-cabeçalho do IP.

3.2.5 É possível capturar toda a troca de mensagens e inclusive capturar o texto passado do cliente para o servidor?

Sim. Como UDP transmite datagramas sem criptografia intrínseca, o conteúdo enviado entre cliente e servidor pode ser capturado e analisado em ferramentas como Wireshark. Nos pacotes analisados:

- Pacotes 4 e 5: contêm dados de Handshake/Ping.
- Pacotes 10 e 11: contêm mensagens de Delete an active session (Bye/Cancel/Error).

O texto transmitido está acessível nos campos de dados do datagrama.

3.2.6 Qual foi a sequência numérica do campo Data em seu teste? Qual o significado?

Nos pacotes analisados:

- Pacotes 4 e 5: dados do Handshake/Ping
- Pacotes 10 e 11: dados do Delete an active session

O campo Data não possui número de sequência próprio no UDP; os valores capturados representam apenas o conteúdo transmitido pelo protocolo de aplicação (RTPproxy neste caso). O significado é a instrução ou comando da aplicação, não um número de controle de transporte.

3.2.7 Qual é o protocolo da camada de transporte nessa troca de mensagens?

O protocolo da camada de transporte usado é:

UDP (User Datagram Protocol)

Isso se confirma pelo fato de que os pacotes analisados não possuem campos de sequência ou confirmação, e o campo `Protocol` do IP é igual a 17.

4 Comparativo entre UDP e TCP

4.1 Quantas mensagens foram trocadas entre o servidor e o cliente em cada um dos protocolos para atingir o mesmo objetivo?

- **UDP:** Nos pacotes analisados (4, 5, 10 e 11), observamos que **quatro mensagens** foram suficientes para realizar as operações de handshake/ping e encerramento da sessão.
- **TCP:** Para atingir o mesmo objetivo (envio e recebimento de dados), é necessário:
 - **Three-way handshake** para estabelecimento de conexão: 3 mensagens.
 - **Troca de dados (camada de aplicação):** pelo menos 2 mensagens (envio e resposta).
 - **Four-way handshake** para encerramento de conexão: 3 mensagens.

Logo, o TCP requer no mínimo **oito mensagens** para completar o mesmo ciclo.

4.2 O que justifica a diferença na quantidade de mensagens trocadas?

A diferença ocorre devido à natureza dos protocolos:

- **UDP:** Não estabelece conexão, não garante entrega, não mantém controle de sequência nem confirmações. Cada datagrama é enviado e recebido independentemente, reduzindo o número de mensagens.
- **TCP:** É orientado à conexão e confiável. O three-way handshake inicial garante sincronização de sequência e abertura segura da conexão; o four-way handshake final garante encerramento ordenado. Cada etapa adiciona mensagens extras à comunicação.

4.3 Discuta as vantagens e desvantagens de cada protocolo

4.3.1 UDP

Vantagens:

- Menor overhead, pois não há handshake nem controle de sequência.
- Mais rápido e eficiente para transmissões curtas ou em tempo real (ex.: VoIP, streaming, jogos online).

Desvantagens:

- Não garante entrega, ordem ou integridade dos pacotes.
- Aplicações precisam lidar com retransmissão ou perda de dados.

4.3.2 TCP

Vantagens:

- Garantia de entrega e ordenação dos dados.
- Controle de congestionamento e confiabilidade.
- Adequado para transferências críticas de dados (ex.: HTTP, FTP, e-mails).

Desvantagens:

- Maior overhead devido a handshakes, confirmações e controle de fluxo.
- Latência maior, podendo ser menos eficiente para aplicações em tempo real.